

Cerințe obligatorii

1. Pattern-urile implementate trebuie să respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificare claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma *cts.nume.prenume.gNrGrupa.denumire_pattern*, *cts.nume.prenume.gNrGrupa.main* (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată gestionării unei linii inteligente de producție dintr-o fabrică.

3p. În cadrul fabricii, evidența materiei prime trebuie gestionată centralizat prin intermediul clasei *RawMaterialManager*, care implementează interfața *IRawMaterialControl*. Sistemul trebuie să păstreze disponibilul curent de materie primă și un istoric al tuturor operațiilor de tip IN și OUT. Fiecare angajat este identificat unic prin *codAngajat* și poate înregistra operații asupra stocului. La recepția de materie primă se va adăuga o cantitate în stoc și se va înregistra o operație de tip IN, iar la utilizarea materiei prime pentru realizarea unui produs se va scădea din stoc cantitatea necesară și se va înregistra o operație de tip OUT.

1.5p. Să se testeze soluția prin:

- efectuarea a cel puțin unei operații de tip IN;
- efectuarea a cel puțin două operații de tip OUT realizate de angajați diferiți;
- încercarea unui consum invalid, care depășește stocul disponibil, situație în care trebuie demonstrată apariția excepției;
- afișarea istoricului complet al operațiilor.

3p. Produsele fabricate pe linia de producție pot avea configurații diferite, în funcție de cerințele clientului. Un obiect de tip *ProductionItem* trebuie configurat flexibil, având câmpuri obligatorii precum: *modelName*, *serialCode*, *materialType*, iar opțional poate include: *batchLabel*, *packagingType*, *specialFinish*, *qualityCheck*, *assemblyInstructions* și *deliveryPriority*. Se cere

implementarea unui mecanism de construire flexibil a obiectelor de tipul `ProductionItem`, fără a permite modificări ulterioare pe obiectele create.

1.5p. Să se testeze soluția prin crearea a cel puțin 3 produse diferite:

- un produs configurat minimal, doar cu elementele obligatorii;
- un produs cu exact 2 opțiuni suplimentare;
- un produs cu numărul maxim permis de opțiuni suplimentare.

```
public interface IRawMaterialControl {  
    void addRawMaterial(String codAngajat, float quantity) throws Exception;  
    void consumeRawMaterial(String codAngajat, String productName, float  
quantity) throws Exception;  
    float getAvailableStock();  
    void displayHistory();  
}
```

```
public interface IProductionItem {  
    String getModelName();  
    String getSerialCode();  
    String getMaterialType();  
    String getBatchLabel();  
    String getPackagingType();  
    void displayInfo();  
}
```